

The Direct Answer

Pivot to Everything Claude Code (ECC) or Superclaude as your primary agent harness, paired with Claude 3.5 Sonnet. [1]

Since you already pay for Claude Pro and Gemini Advanced, you have immediate access to the highest-performing models on the market. Do not throttle your setup by running low-tier models locally on your NPU. Use your Qualcomm NPU exclusively for local embedding generations, small utility task models (via Llama.cpp), or vector database indexing. [1]

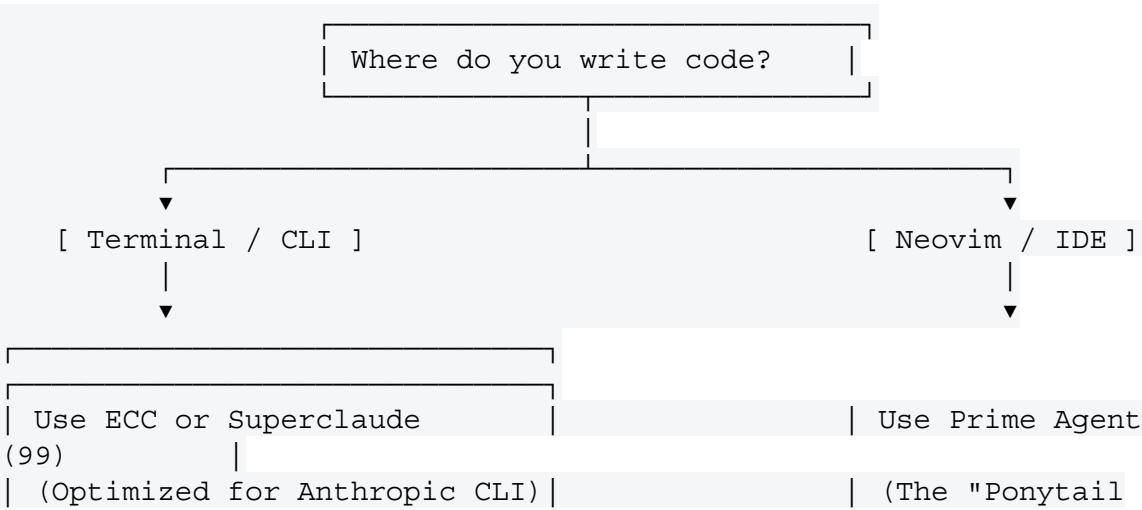
1. The New Architecture: Framework vs. MCP

Your paid API keys completely change how your frameworks interact with your Model Context Protocol (MCP) servers.

- **No "Cancellation" Effect:** Frameworks like ECC and Superclaude are **MCP Clients**. They do not replace your servers; they are built specifically to *consume* them.
- **The Hub & Spoke Model:** ECC or Superclaude will sit at the center (the Hub). Your 9+ GitHub MCP servers (Perplexity, Supabase, Linear, Playwright) will plug directly into it (the Spokes).
- **The Workflow:** When you ask ECC to debug a repository, it orchestrates the task, uses your **Perplexity MCP** to search the live web for recent framework updates, and uses **Playwright** to test the UI.

2. Selecting Your Framework (ECC vs. Superclaude vs. Pi)

Since you have Claude Pro, pick your harness based on your development environment:



```
Caveman" tool) |  
|  
|
```

- **Choose ECC if:** You want a seamless drop-in wrapper around claude-code. It is designed to optimize token usage, minimize costly loops, and manage tool definitions specifically for Anthropic's ecosystem.
- **Choose Superclaude if:** You want a highly customized system prompt that bypasses standard safety guardrails for deeper systems-level debugging.
- **Downgrade Pi Agent:** Move Pi Agent to your backup tier. It is great for local-first setups, but it doesn't leverage the deep tool-use optimizations inherent in Claude-specific harnesses.

3. Strategic Integration Blueprint

Do not try to spin up all 30 repositories simultaneously. Build your local ecosystem in three specific phases.

Phase 1: The Core Control Loop (Day 1)

1. **Harness:** Initialize **ECC** in your Termux or desktop environment.
2. **Model:** Bind it to **Claude 3.5 Sonnet** using your Claude Pro API credentials.
3. **Memory:** Plug **mem0** or your **Reasoning Bank** mapping directly into the system prompt configuration of ECC. This gives the agent a persistent memory layer across terminal sessions.

Phase 2: The Infrastructure Layer (Day 2)

1. **Data Persistence:** Launch **Supabase** locally via Docker or their CLI to act as your centralized long-term vector database.
2. **Context Loading:** Use **Firecrawl** and **Jina Launcher** to ingest clean, markdown-formatted documentation from your Unslaught, Google Dev, and Qualcomm SDK directories straight into Supabase.
3. **Token Management:** Wrap your context inputs in **RTK-AI** (Rust Token Killer) before passing them to Claude to prevent massive context-window bloat and save your API rate limits.

Phase 3: The Automation Layer (Day 3+)

1. **Browser Control:** Hook the **Playwright** and **Browser-use** MCP servers into ECC. This allows Claude to physically open a browser window, navigate local web UI tools, and test your code live.
2. **Multi-Agent Escapes:** If a task is too large for ECC alone, use **CrewAI** or **Ringer** to split the goal into a multi-agent swarm (e.g., Agent 1 writes code, Agent 2 runs Semgrep to audit it, Agent 3 refactors).

Next Steps to Advance Your Build

To help configure your initialization files correctly, let me know:

- Are you primarily developing inside **Termux on an Android device**, or on a **Qualcomm Snapdragon laptop**?
- Do you prefer working directly inside a **standard terminal command line**, or are you a **Neovim** user?

[1] <https://www.reddit.com>